```
# ==========================================
# 1. IMPORT LIBRARIES
# ==========================================
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.sequence import pad_sequences


# ==========================================
# 2. LOAD DATASET (IMDB)
# ==========================================
vocab_size = 10000

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.imdb.load_data(num_words=vocab_size)

print("Training samples:", len(x_train))
print("Testing samples:", len(x_test))


# ==========================================
# 3. PREPROCESSING
# ==========================================
max_len = 200

x_train = pad_sequences(x_train, maxlen=max_len, padding='post')
x_test  = pad_sequences(x_test,  maxlen=max_len, padding='post')


# ==========================================
# 4. BUILD RNN MODEL (FIXED)
# ==========================================
model = models.Sequential([

    #  Input layer (fixes unbuilt issue)
    layers.Input(shape=(max_len,)),

    # Embedding layer
    layers.Embedding(input_dim=vocab_size, output_dim=128),

    #  Simple RNN layer
    layers.SimpleRNN(64),

    # Dense layer
    layers.Dense(64, activation='relu'),

    # Dropout
    layers.Dropout(0.5),

    # Output layer
    layers.Dense(1, activation='sigmoid')
])

# Now summary will show correct parameters
model.summary()


# ==========================================
# 5. COMPILE MODEL
# ==========================================
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)


# ==========================================
# 6. TRAIN MODEL
# ==========================================
history = model.fit(
    x_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2
)


# ==========================================
# 7. EVALUATE MODEL
# ==========================================
loss, accuracy = model.evaluate(x_test, y_test)
```

```
loss, accuracy = model.evaluate(x_test, y_test)

print("\nTest Accuracy:", accuracy * 100)


# ==========================================
# 8. PREDICT NEW TEXT
# ==========================================
word_index = tf.keras.datasets.imdb.get_word_index()

def encode_text(text):
    words = text.lower().split()
    encoded = [word_index.get(word, 2) for word in words]
    padded = pad_sequences([encoded], maxlen=max_len, padding='post')
    return padded


def predict_sentiment(text):
    processed = encode_text(text)
    prediction = model.predict(processed)[0][0]

    print("\nReview:", text)

    if prediction >= 0.5:
        print("Sentiment: Positive 😊")
    else:
        print("Sentiment: Negative 😞")


# Example predictions
predict_sentiment("This movie was excellent and amazing")
predict_sentiment("This movie was very bad and boring")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training samples: 25000
Testing samples: 25000
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 200, 128)	1,280,000
simple_rnn (SimpleRNN)	(None, 64)	12,352
dense (Dense)	(None, 64)	4,160